SentryScan

Autonomous Multi-Agent Cyber Security Platform

Project Status, Architecture, Roadmap (v2.1, Nov 2026)

1. Executive summary

SentryScan senses fleet telemetry, verifies it against poisoning, lets specialised agents negotiate (Contract-Net Protocol) who handles each case, deliberates with a BDI model, and contains threats through a remediation actuator. All inter-agent messages are Ed25519-signed. Every event is recorded in a JSONL audit stream and a SQLite store that feeds a web console.

The platform ships as two products plus a demo rig: the Executive Portal (**/**), the Master Cyber Console (**/master**) and the Demo Control Center (**/control**).

2. Architecture

Data path: Fleet Sensors → Integrity Gate → Supervisor Graph (LangGraph + CNP auction) → VIGIL Collective (Scout, Triage, BDI Analyst) → Containment Actuator. Side paths: Zero-Trust Gate, Knowledge Enricher (ontology + NVD), LLM factory, Run Store, Dashboard Gateway (FastAPI + WebSocket), React console.

Component	Module	Role
Fleet Sensors	simulation/fleet.py	Simulated workstation and network telemetry
Integrity Gate	integrity/checker.py	Detects poisoned or malformed telemetry
Supervisor Graph	graphs/supervisor.py	State machine and Contract-Net auction
BDI Analyst	agents/analyst.py, bdi/	Beliefs, desires, intentions, reconsideration
Remediator	agents/remediator.py	Host isolation / quarantine actions
Zero-Trust Gate	security/gate.py	Signature, nonce, replay, recipient checks
Knowledge Enricher	knowledge/enricher.py	Ontology and CVE enrichment
Run Store	storage/store.py	SQLite WAL relational store
Recorder	observability/recorder.py	Gap-free JSONL event stream
Gateway	dashboard_gateway/server.py	REST + WebSocket for the console

3. What has been built

Agents and reasoning

• Scout, Triage, Analyst, Remediator and Reflex agents.

• BDI deliberation (beliefs, desires, intentions) with reconsideration tests.

• Weighted optimizer with formula, engine, brute-force and edge-case tests.

Coordination and security

• Contract-Net negotiation protocol with proposal roles and scoring.

• Ed25519 identity, canonical serialisation, registry, nonce cache.

• Rejection of forged, tampered, unknown, wrong-recipient, stale and replayed messages.

Persistence and delivery

• Dual-sink persistence: JSONL authoritative stream and SQLite tables (events, telemetry, messages, findings, bdi_deliberations, pending_quarantines, llm_calls).

• Loopback WebSocket gateway with resync by sequence; prebuilt dashboard bundle; Rich CLI fallback.

• One-command demo, replay and parity tests, human-traffic and poisoning simulators, ground-truth evaluator.

• Concurrency layer: queue manager, daemon, metrics and stats.

• Socket and in-process transports with framing.

4. Run modes and log locations

Mode	Command	Output directory
Continuous live SOC	python main.py --live	data/live/run-live.jsonl + .db
Capstone demo	python main.py --demo	data/runs/capstone_demo/run-capstone-0042.*
Human traffic test	python main.py --test-human	data/runs/human_test/run-human-test-0042.*
Evaluation harness	python main.py --eval	data/runs/eval/run-eval-s{seed}-c{cycles}.*

Each event records event_id/seq, timestamp_utc, component, event_type (RECORD_RECEIVED, RECORD_REJECTED, MESSAGE_SENT, FINDING_CREATED, STATE_TRANSITION, PARAMS_UPDATE) and details.

5. Recent change history

Commit	Summary
721dfdd	Demo Control Center, POLARIS compliance rebuild, SQLite concurrency optimisation, README
a5267d0	CI fixes (lint, test deps, secret false-positives), dual-product README
d9b8b37	v2.0: VIGIL SOC, SENTRA Data Discovery, POLARIS Compliance
7d0aedb	Merge milestone 5.2 capstone and dashboard
92dc6b6 – f113be8	5.2: SQLite/JSONL persistence, WebSocket gateway, Rich dashboard, bundle, one-command demo
32f34ef	Merge milestone 5.1 message authentication

6. Quality evidence

• About 85 source modules and about 50 test files (counted from the repository listing).

• Integration tests: multi-agent coordination, BDI reconsideration, knowledge enrichment, capstone parity and replay.

• Unit tests span security gate, signing, identity, store, transport, supervisor graph, dashboard gateway.

7. Remaining work

Item	Detail	Priority
Real sensor ingestion	Agent, syslog or EDR adapters instead of simulator	High
Live containment	Actuators are dry-run; add approval flow and rollback	High
Console authN/authZ	Users, roles, operator audit, agent key rotation	High
Packaging	Docker/compose, config management, retention, metrics export	Medium
Live NVD cache	Scheduled refresh and offline fallback	Medium

8. Refinement backlog

Area	Issue	Action
Large modules	store.py ~41 KB, server.py ~49 KB	Split by concern (queries, writers, routes)
Evaluation	Limited scenarios	Per-attack precision/recall report
LLM layer	Failure and cost handling	Timeouts, retries, spend from llm_calls, prompt regression tests
UI	Polish gaps	Accessibility, empty/error states, WebSocket load tests
Docs	Scattered	Single architecture and operations guide

9. Roadmap (proposed)

• **Phase 1 Harden:** split modules, console auth, CI quality gates.

• **Phase 2 Connect:** real sensor adapters, live NVD cache.

• **Phase 3 Act:** approved live containment with rollback.

• **Phase 4 Ship:** containers, operations docs, benchmark report.

Note: counts and maturity estimates derive from file listings, git log and the architecture diagram; no test run was executed for this report. Roadmap items are proposals.